

Trabajo Práctico 02

Contenedores: la app del semestre



Alumno	Marcos Don
Matrícula	2319274
Repositorio	github.com/marcosdon28/IngeSoft3-2319274
Aplicación	Inventario de productos
Stack	FastAPI + React/Vite + PostgreSQL
Imágenes	ghcr.io/marcosdon28/inventario- {backend,frontend}:v0.1.0
Versión entregada	v2.0.0
Fecha	26/08/2026

1. Objetivo y qué se evalúa

Contenerizar la aplicación que va a acompañar toda la materia: Dockerfiles multi-etapa para backend y frontend, orquestación completa con compose, e imágenes publicadas en un registry. El escenario del enunciado lo resume bien: dejar el proyecto en un estado donde **cualquiera que clone el repositorio levante el sistema completo con un comando**, y donde las imágenes estén publicadas para que otros entornos las consuman.

Requisito	Estado
App full-stack (front + back + base) corriendo, justificada en <code>decisiones.md</code>	✓ Inventario de productos, elegida contra los 5 criterios
Dockerfile multi-stage para back y front, cada uno con su <code>.dockerignore</code>	✓ 1,62 GB → 326 MB y 229 MB → 93 MB
<code>nginx.conf</code> con el proxy hacia el backend	✓ con <code>resolver</code> y <code>proxy_pass</code> por variable
Compose con volumen, red por nombre, <code>depends_on</code> + <code>healthcheck</code> , secreto por <code>.env</code>	✓
Imágenes en registry gratuito, tag semver, visibilidad pública	✓ ghcr.io, <code>v0.1.0</code> , públicas
<code>docker-compose.registry.yml</code> probado de verdad	✓ probado deslogueado
<code>README.md</code> con los pasos de arranque desde cero	✓

La guía se hace dos veces. Primero sobre el sample de la cátedra —para practicar, y eso no se entrega— y después sobre la app propia, que es lo que se defiende. Hice las dos, en ese orden, y la §4 de este informe cuenta qué me dejó la primera.

2. La aplicación elegida

Un sistema de inventario de productos, escrito por mí: FastAPI (Python 3.12) + React/Vite + PostgreSQL 16. La escribí en vez de adoptar un proyecto de GitHub porque el quinto criterio de la guía de selección —*entenderla lo suficiente para modificarla en vivo*— se cumple de verdad sólo cuando el código es propio: en la mesa del Integrador hay que hacer un cambio delante del profesor.

Criterio de elegir-app.md	Cómo lo cumple
1. Que corra hoy	<code>cp .env.example .env && docker compose up -d --build</code> . No necesita Python, Node ni PostgreSQL instalados
2. Conocer los comandos de build	<code>pip install -r requirements.txt + uvicorn; npm ci + npm run build</code> . Son las líneas de los Dockerfiles
3. Dónde se configura la base	En una variable: <code>DATABASE_URL</code> , leída en <code>app/config.py</code> . Nunca hardcodeada
4. Lógica para testear	7 reglas de negocio con casos válidos, inválidos y de borde — el TP5 pide 8 tests de back y 4 de front
5. Entenderla para modificarla	Código propio, en capas explícitas

Tamaño: cuatro pantallas y tres entidades. La guía avisa que más grande no suma nota — sólo builds más lentos y más puntos de falla.

Las 7 reglas de negocio

#	Regla	Por qué existe
1	Una salida no puede superar el stock disponible	El stock son unidades físicas: nunca puede quedar negativo
2	El SKU es único	Dos productos con el mismo SKU hacen imposible saber cuál se movió
3	No se puede eliminar una categoría con productos	Evita productos huérfanos; borrar en cascada destruiría datos sin avisar
4	Precio y stock no negativos; cantidad > 0	Validación de dominio en el borde de entrada
5	Descuento por cantidad desde un umbral	Cálculo con bordes — el umbral es inclusive
6	<code>stock <= stock_minimo</code> marca bajo stock	Propiedad derivada , no columna: no puede desincronizarse
7	Un producto inactivo no admite movimientos	Si igual se le cargan movimientos, la baja es decorativa

Un caso de borde que apareció al probar la regla 5: con umbral en 10 unidades y 10 % de descuento, **llevar 9 cuesta lo mismo que llevar 10** ($9 \times 1800 = 16.200$ y $10 \times 1800 \times 0,9 = 16.200$). No es un error —es lo que la regla dice— pero es exactamente el tipo de cosa que un test de borde expone y una lectura del código no.

3. Arquitectura: MVC en FastAPI

Capa	Dónde vive	Qué sabe
Model	<code>app/models/</code> (SQLAlchemy) + <code>app/schemas/</code> (Pydantic)	Las entidades, su persistencia y el contrato de la API
Controller	<code>app/routers/</code> + <code>app/services/</code>	Los routers traducen HTTP; los services tienen las reglas
View	<code>frontend/src/</code>	Presentación; las reglas de la vista aisladas en <code>lib/reglas.js</code>

La decisión que más importa de todo el diseño: **los `services/` no importan FastAPI**. Lanzas excepciones propias (`ReglaDeNegocioError`) y los routers las traducen a códigos HTTP. Gracias a eso una regla se testea llamando a una función, sin levantar un servidor — que es exactamente lo que va a necesitar el TP5. Lo mismo del lado del frontend: `lib/reglas.js` son funciones puras, sin React ni `fetch` adentro.

```
backend/app/  
  models/      SQLAlchemy: Categoria, Producto, Movimiento  
  schemas/     Pydantic: el contrato de entrada y salida  
  services/    ← las 7 reglas viven acá, sin HTTP  
  routers/     los endpoints: traducen HTTP a llamadas a services  
  config.py    DATABASE_URL y los parámetros del descuento, del entorno
```

4. La primera pasada: el sample de la cátedra

Antes de tocar mi app hice la guía completa sobre `demo-fullstack` (.NET 8 + React + PostgreSQL), que es lo que el enunciado pide y que en su momento parece una pérdida de tiempo. No lo fue: los tres problemas que me frenaron ahí volvieron a aparecer en mi app, y la segunda vez ya sabía qué eran.

Lo que me dejó

- **La diferencia SDK / runtime, con números.** La imagen que compila (`dotnet/sdk:8.0`) pesa **1,25 GB**; la que sólo ejecuta (`dotnet/aspnet:8.0`), 350 MB. Ver esos dos números al lado explica el multi-stage mejor que cualquier definición.
- **El 502 del frontend sin red compartida.** Con el contenedor del front y el del back corriendo por separado, `curl localhost:3000/api/tareas` devuelve **502**: existen los dos, pero no se conocen — el nombre `backend` del `nginx.conf` no resuelve. Eso es exactamente el problema que resuelve `compose`, y verlo fallar primero hace que el `docker-compose.yml` deje de ser magia.
- **El orden `Waiting` → `Healthy` → `Starting`** en la salida de `compose`: la diferencia entre "el contenedor arrancó" y "el servicio está listo", que es lo que separa `depends_on` a secas de `depends_on` + `healthcheck`.

Un obstáculo que no era de Docker: el sample es .NET y yo no tenía el SDK. `brew install --cask dotnet-sdk` pide contraseña de administrador; lo resolví con el script oficial `dot.net/v1/dotnet-install.sh --channel 8.0`, que instala en `~/dotnet` sin privilegios. Detalle que conviene entender: para *construir la imagen* el SDK local no hace falta —vive dentro del contenedor— pero sí para el paso de correr la app nativa antes de contenerizarla.

5. Los Dockerfiles multi-etapa

Backend

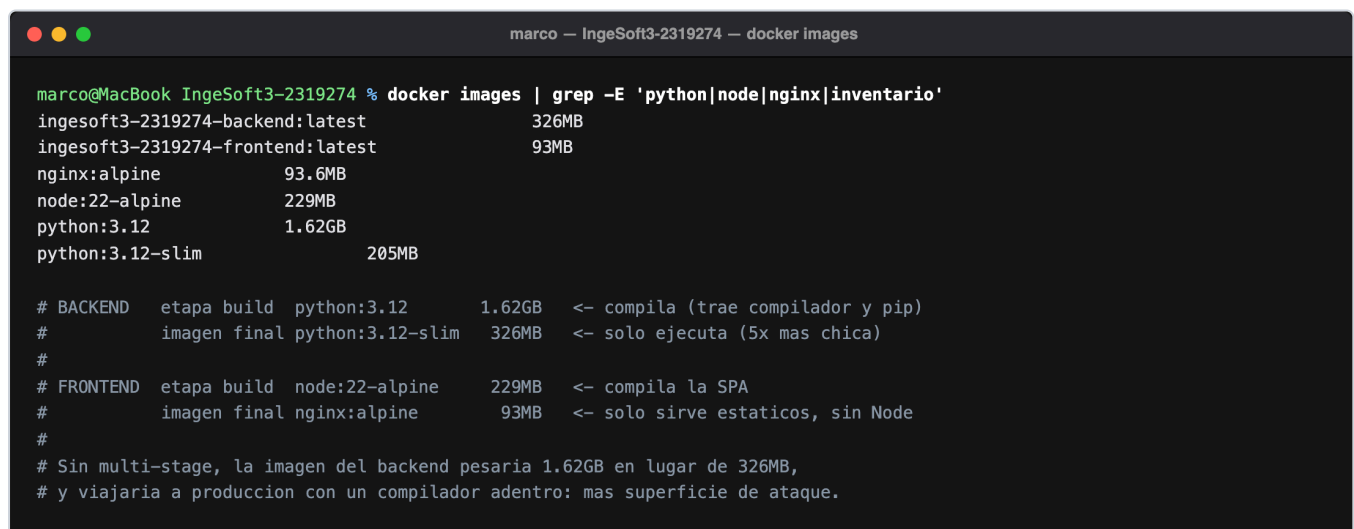
```
# ---- Etapa 1: build (imagen completa: compilador y herramientas) ----
FROM python:3.12 AS build
COPY requirements.txt .                # las dependencias PRIMERO...
RUN python -m venv /opt/venv \
    && /opt/venv/bin/pip install -r requirements.txt

# ---- Etapa 2: runtime (mínima, sin compilador ni cache de pip) ----
FROM python:3.12-slim AS final
COPY --from=build /opt/venv /opt/venv  # sólo el venv ya resuelto
ENV PATH="/opt/venv/bin:$PATH"
COPY app ./app                        # ...y el código ÚLTIMO
RUN useradd --create-home --uid 1001 appuser
USER appuser                          # no corre como root
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dos reglas de oro, las dos visibles ahí: **el archivo de dependencias se copia antes que el código**, porque cuando una capa cambia Docker invalida esa capa y todas las siguientes — con este orden, tocar una línea de código no vuelve a instalar nada; y **el SDK no viaja a producción**: la etapa final sólo recibe el virtualenv ya resuelto.

Frontend

Mismo patrón con otro argumento, todavía más claro: una SPA compilada son **archivos estáticos**. Node compila en la primera etapa y no aparece en la segunda, donde sólo hay nginx y los archivos de `dist/`.



```
marco@MacBook IngeSoft3-2319274 % docker images | grep -E 'python|node|nginx|inventario'
ingesoft3-2319274-backend:latest          326MB
ingesoft3-2319274-frontend:latest        93MB
nginx:alpine                             93.6MB
node:22-alpine                           229MB
python:3.12                              1.62GB
python:3.12-slim                          205MB

# BACKEND  etapa build  python:3.12      1.62GB  <- compila (trae compilador y pip)
#          imagen final python:3.12-slim 326MB   <- solo ejecuta (5x mas chica)
#
# FRONTEND etapa build  node:22-alpine   229MB   <- compila la SPA
#          imagen final nginx:alpine     93MB    <- solo sirve estaticos, sin Node
#
# Sin multi-stage, la imagen del backend pesaria 1.62GB en lugar de 326MB,
# y viajaria a produccion con un compilador adentro: mas superficie de ataque.
```

Los tamaños medidos, no estimados. Sin multi-stage la imagen del backend pesaría 1,62 GB en vez de 326 MB — y viajaría a producción con un compilador adentro, que es superficie de ataque que no hace falta.

Los `.dockerignore` son dos, no uno. Docker los busca en el **contexto** que se le pasa (`./backend` y `./frontend`), no en la raíz del repositorio. Y tienen contenidos distintos: el del backend excluye `__pycache__/` y `.venv/` ; el del frontend, `node_modules/` — que no es opcional, porque si no se excluye, el `COPY` . . pisa los módulos que `npm ci` acaba de instalar para Linux con los de la máquina local.

6. Cómo se hablan los tres servicios

Tres ámbitos distintos, y confundirlos explica casi todos los problemas de red de este práctico.

Camino	Cómo se resuelve
backend → db	Por el nombre del servicio (<code>@db:5432</code>). Compose crea una red interna con DNS embebido: no importa en qué IP cayó la base
host → backend	Sólo por los puertos publicados (<code>8000:8000</code>). La base no publica ninguno : no es accesible desde afuera
browser → backend	El caso trampa: el JavaScript de la SPA corre en el browser , que vive en mi máquina y no en la red de compose. <code>http://backend:8000</code> no resolvería nunca

Para el tercero elegí la solución (a) de la guía: la SPA llama a **rutas relativas** (`/api/...`), y quien las traduce es el servidor de Vite en desarrollo y **nginx** en el contenedor. Dos consecuencias, y las dos importan más adelante: la **misma imagen sirve en cualquier entorno** porque no tiene ninguna URL compilada adentro (eso es lo que va a hacer posible el TP6), y como para el browser todo sale del mismo origen, **no hay CORS que configurar**.

```
location /api/ {
    resolver 127.0.0.11 valid=10s ipv6=off;    # el DNS de Docker
    set $backend_api http://backend:8000;      # ← el nombre en una VARIABLE
    proxy_pass $backend_api;                  # ← y SIN barra al final
}
```

Los dos detalles del bloque parecen de más y no lo son. **El nombre va en una variable** porque con el nombre escrito directo nginx lo resuelve *al arrancar* y, si el backend todavía no existe, se niega a levantar con `host not found in upstream` — o sea que el contenedor del frontend no podría correr solo. Y el **proxy_pass va sin barra final**: con barra, nginx reescribe el prefijo y manda `/api/productos` a `/productos`, con lo que la API devuelve 404 en todas las llamadas.

7. El sistema completo con un comando

```
marco — IngeSoft3-2319274 — docker compose up desde cero

marco@MacBook IngeSoft3-2319274 % docker compose down -v
Network ingesoft3-2319274_default Removing
Volume ingesoft3-2319274_db_data Removed
Network ingesoft3-2319274_default Removed

marco@MacBook IngeSoft3-2319274 % cp .env.example .env
marco@MacBook IngeSoft3-2319274 % docker compose up -d --build
Network ingesoft3-2319274_default Creating
Network ingesoft3-2319274_default Created
Volume ingesoft3-2319274_db_data Created
Container ingesoft3-2319274-db-1 Created
Container ingesoft3-2319274-backend-1 Created
Container ingesoft3-2319274-frontend-1 Created
Container ingesoft3-2319274-db-1 Starting
Container ingesoft3-2319274-db-1 Started
Container ingesoft3-2319274-db-1 Waiting
Container ingesoft3-2319274-db-1 Healthy
Container ingesoft3-2319274-backend-1 Starting
Container ingesoft3-2319274-backend-1 Started
Container ingesoft3-2319274-frontend-1 Starting
Container ingesoft3-2319274-frontend-1 Started

marco@MacBook IngeSoft3-2319274 % docker compose ps
SERVICE      STATUS              PORTS
backend        Up Less than a second 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
db             Up 5 seconds (healthy) 5432/tcp
frontend       Up Less than a second 0.0.0.0:3000->80/tcp, [::]:3000->80/tcp

marco@MacBook IngeSoft3-2319274 % curl -s localhost:8000/health
{"status":"ok"}

marco@MacBook IngeSoft3-2319274 % ./scripts/datos-demo.sh
Esperando a que la API responda...
Categorías...
Productos...
Movimientos...
Listo. Abrió http://localhost:3000

marco@MacBook IngeSoft3-2319274 % curl -s localhost:3000/api/productos # front -> nginx -> backend -> db
BEB-002  Agua mineral 500ml    stock= 32  bajo_stock=False
LIM-001  Detergente 750ml     stock= 15  bajo_stock=False
BEB-001  Gaseosa 1.5L        stock= 21  bajo_stock=False
ALM-001  Yerba mate 1kg       stock= 25  bajo_stock=False

# El browser pide /api/productos al puerto 3000 (nginx), nginx lo reenvía
# a backend:8000 por la red interna, y el backend consulta a db:5432.
# Los tres servicios se encuentran POR NOMBRE. Ningun CORS de por medio.
```

Arranque desde cero, empezando por `down -v` para no arrastrar estado. En el medio se ve `db Waiting → db Healthy → backend Starting`: el backend no arranca hasta que PostgreSQL acepta conexiones.

Por qué `depends_on` no alcanza

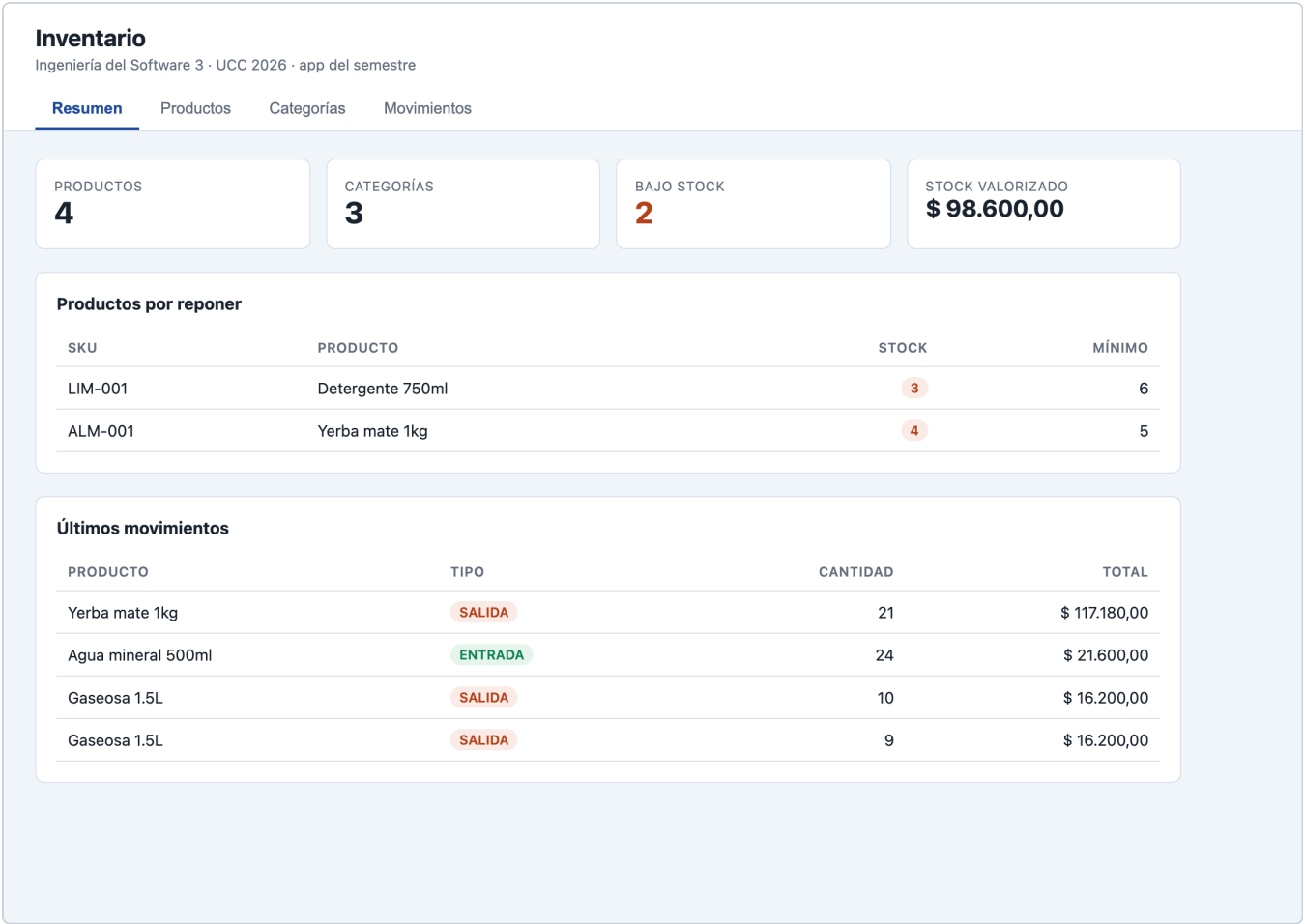
`depends_on` garantiza el orden de **arranque**, no de **disponibilidad**. Que el contenedor de PostgreSQL haya arrancado no significa que acepte conexiones — hay varios segundos entre una cosa y la otra, y en ese hueco el backend se conecta, falla y se muere. Por eso el servicio `db` declara un `healthcheck` con `pg_isready` y el backend espera con `condition: service_healthy`.

La distinción "arrancó" contra "está listo" reaparece en cada sistema distribuido que uno toque.

Los secretos

La contraseña de la base sale de `${DB_PASSWORD}`, que vive en un `.env` **que no se commitea**. Lo que sí se versiona es `.env.example`, que documenta qué variables hacen falta sin traer ningún valor real. Por eso el arranque son **dos comandos y no uno** — y eso no es un defecto de la entrega: es el precio de que el secreto no viaje en el repositorio. En el TP4 esos secretos migran a los secrets de la plataforma de CI.

8. La aplicación funcionando



Inventario

Ingeniería del Software 3 · UCC 2026 · app del semestre

Resumen Productos Categorías **Movimientos**

Registrar movimiento

Producto

Elegir...

Tipo

Entrada

Cantidad

Registrar

Elegí un producto.

Historial

FECHA	PRODUCTO	TIPO	CANTIDAD	DESCUENTO	TOTAL
26/8/2026, 03:22:40	Yerba mate 1kg	SALIDA	21	10%	\$ 117.180,00
26/8/2026, 03:22:40	Agua mineral 500ml	ENTRADA	24	—	\$ 21.600,00
26/8/2026, 03:22:40	Gaseosa 1.5L	SALIDA	10	10%	\$ 16.200,00
26/8/2026, 03:22:40	Gaseosa 1.5L	SALIDA	9	—	\$ 16.200,00

La regla 5 visible: la salida de 10 unidades lleva 10 % de descuento y la de 9 no. Arriba, el botón deshabilitado con el motivo escrito — las reglas 1, 4 y 7 evaluadas en vivo.

La validación de la interfaz es **comodidad, no seguridad**: el backend rechaza igual cada operación inválida, con un 400 y el motivo. Las dos capas dicen lo mismo porque las dos leen la misma regla, no porque estén duplicadas a mano — en el front vive en `lib/reglas.js`, en el back en `services/`.

Inventario

Ingeniería del Software 3 · UCC 2026 · app del semestre

Resumen Productos **Categorías** Movimientos

Nueva categoría

Nombre

Bebidas

Agregar

Categorías

NOMBRE	PRODUCTOS	
Almacen	1	Eliminar
Bebidas	2	Eliminar
Limpieza	1	Eliminar

La regla 3 reflejada en la vista: el botón *Eliminar* está deshabilitado cuando la categoría tiene productos asociados.

9. Persistencia: qué sobrevive y qué no

Los contenedores son **efímeros por diseño**: la capa de escritura muere con el contenedor. Eso es una ventaja para la aplicación —contenedores descartables ⇒ deploys y rollbacks triviales— y una catástrofe para los datos. La solución es separar el estado explícitamente: el directorio de datos de PostgreSQL vive en el volumen **nombrado** `db_data`.

```
marco — IngeSoft3-2319274 — prueba de persistencia

marco@MacBook IngeSoft3-2319274 % curl -s localhost:8000/api/productos | jq -r '.[0] | "{sku: .sku, stock: .stock}"'
BEB-002 stock=32
LIM-001 stock=15
BEB-001 stock=21
ALM-001 stock=25

marco@MacBook IngeSoft3-2319274 % docker compose down && docker compose up -d
Network ingesoft3-2319274_default Removing
Network ingesoft3-2319274_default Removed
Container ingesoft3-2319274-frontend-1 Starting
Container ingesoft3-2319274-frontend-1 Started

marco@MacBook IngeSoft3-2319274 % curl -s localhost:8000/api/productos | jq ...
BEB-002 stock=32
LIM-001 stock=15
BEB-001 stock=21
ALM-001 stock=25
# → LOS DATOS SIGUEN: el volumen sobrevivio a la destruccion de los contenedores

marco@MacBook IngeSoft3-2319274 % docker compose down -v && docker compose up -d
Volume ingesoft3-2319274_db_data Removed
Network ingesoft3-2319274_default Removed
Container ingesoft3-2319274-frontend-1 Starting
Container ingesoft3-2319274-frontend-1 Started

marco@MacBook IngeSoft3-2319274 % curl -s localhost:8000/api/productos
[]
# → VACIO: -v se llevo tambien el volumen. 'down' apaga; 'down -v' ademas OLVIDA.
```

Los stocks antes y después. `down` destruye los contenedores y los datos siguen; `down -v` se lleva también el volumen y la lista vuelve vacía.

	Cómo se escribe	Dónde viven los datos	Cuándo usarlo
Volumen nombrado	<code>db_data:/var/lib/postgresql/data</code>	En un área que administra Docker (en Mac, dentro de su VM)	Bases de datos — es lo que usa este TP
Bind mount	<code>./datos:/var/lib/postgresql/data</code>	En una carpeta mía, que puedo abrir	Meter código o configuración, ver cambios en vivo

La diferencia visual es un carácter: si lo de la izquierda empieza con `./` es una ruta; si es una palabra suelta, es un volumen de Docker. Para la base va volumen nombrado: en Mac hay una máquina virtual en el medio, así que un bind mount del directorio de datos de PostgreSQL es notablemente más lento y da problemas de permisos.

`down` apaga; `down -v` además **olvida**.

10. Publicar en el registry

Un **registry** es a las imágenes lo que GitHub es al código. Publiqué en **GitHub Container Registry** (ghcr.io): la cuenta ya existe —es la del TP1—, las imágenes quedan junto al código, y en el TP7 el pipeline va a poder autenticarse contra ghcr **sin secretos**, usando el `GITHUB_TOKEN` del propio workflow.

```
gh auth refresh -h github.com -s write:packages # el scope no venía del TP1
gh auth token | docker login ghcr.io -u marcosdon28 --password-stdin
docker tag ingesoft3-2319274-backend ghcr.io/marcosdon28/inventario-backend:v0.1.0
docker push ghcr.io/marcosdon28/inventario-backend:v0.1.0
```

El docker login miente. Da `Login Succeeded` con cualquier token válido, tenga o no permiso sobre packages — y recién falla el `push`, con `denied`. La única comprobación real de la credencial es empujar algo.

The screenshot shows the Docker Hub interface for a package named 'inventario-backend' version 'v0.1.0'. The package is marked as 'Public' and 'Latest'. The 'Installation' tab is active, showing the command to pull the image: `$ docker pull ghcr.io/marcosdon28/inventario-backend:v0.1.0`. The 'Recent tagged image versions' section shows the current version 'v0.1.0' published 10 minutes ago. A section titled 'Link this package to a repository' provides instructions on how to connect the package to a repository, including a 'Connect Repository' button and a note about linking via Dockerfile. The footer shows the GitHub logo and copyright information for 2026.

El package con la etiqueta **Public**, el tag semver `v0.1.0` y el comando exacto para bajarlo. Los packages de ghcr **nacen privados**: hubo que cambiarles la visibilidad una por una.

La prueba de verdad: bajarlas sin credenciales

Decir que una imagen está pública es fácil. La prueba es bajarla estando deslogueado — y para que la prueba signifique algo hay que borrar antes **las tres copias locales**, porque Docker no guarda imágenes sino **capas identificadas por su contenido**.


```

marco — IngeSoft3-2319274 — pull anónimo desde ghcr

marco@MacBook IngeSoft3-2319274 % docker compose down --rmi local
Network ingesoft3-2319274_default Removing
Image ingesoft3-2319274-frontend:latest Removed
Image ghcr.io/marcosdon28/inventario-backend:v0.1.0 Removed
Image ghcr.io/marcosdon28/inventario-frontend:v0.1.0 Removed
Image ingesoft3-2319274-backend:latest Removed
Network ingesoft3-2319274_default Removed

marco@MacBook IngeSoft3-2319274 % docker rmi ghcr.io/marcosdon28/inventario-backend:v0.1.0 ghcr.io/marcosdon28/inventario-frontend:v0.1.0
Error response from daemon: No such image: ghcr.io/marcosdon28/inventario-backend:v0.1.0
Error response from daemon: No such image: ghcr.io/marcosdon28/inventario-frontend:v0.1.0

marco@MacBook IngeSoft3-2319274 % docker builder prune -af # el cache tambien guarda capas
1jl1ev17ny0sattn6vuf1bhnu      true    135.9MB    31 minutes ago
Total: 40.15GB

marco@MacBook IngeSoft3-2319274 % docker logout ghcr.io # dejo de estar autenticado
Removing login credentials for ghcr.io

marco@MacBook IngeSoft3-2319274 % docker pull ghcr.io/marcosdon28/inventario-backend:v0.1.0
58692fb03823: Pull complete
96342d02428d: Pull complete
597155223672: Pull complete
Digest: sha256:f1ea395502658aec8f959baff395911b945a567f344adb97ba4bb4f6dba55c82
Status: Downloaded newer image for ghcr.io/marcosdon28/inventario-backend:v0.1.0
ghcr.io/marcosdon28/inventario-backend:v0.1.0

marco@MacBook IngeSoft3-2319274 % docker compose -f docker-compose.registry.yml up -d # NO construye: descarga
Network ingesoft3-2319274_default Created
Container ingesoft3-2319274-db-1 Created
Container ingesoft3-2319274-backend-1 Created
Container ingesoft3-2319274-frontend-1 Created
Container ingesoft3-2319274-db-1 Starting
Container ingesoft3-2319274-db-1 Started
Container ingesoft3-2319274-db-1 Waiting
Container ingesoft3-2319274-db-1 Healthy
Container ingesoft3-2319274-backend-1 Starting
Container ingesoft3-2319274-backend-1 Started
Container ingesoft3-2319274-frontend-1 Starting
Container ingesoft3-2319274-frontend-1 Started

marco@MacBook IngeSoft3-2319274 % docker compose -f docker-compose.registry.yml ps
SERVICE      IMAGE                                     STATUS
backend        ghcr.io/marcosdon28/inventario-backend:v0.1.0 Up Less than a second
db              postgres:16-alpine                     Up 5 seconds (healthy)
frontend        ghcr.io/marcosdon28/inventario-frontend:v0.1.0 Up Less than a second

marco@MacBook IngeSoft3-2319274 % curl -s localhost:8000/health && ./scripts/datos-demo.sh
{"status":"ok"}
Esperando a que la API responda...
Categorías...

marco@MacBook IngeSoft3-2319274 % curl -s localhost:3000/api/productos
BEB-002  Agua mineral 500ml    stock= 32
LIM-001  Detergente 750ml      stock= 3
BEB-001  Gaseosa 1.5L          stock= 21
ALM-001  Yerba mate 1kg        stock= 4

# Sistema completo corriendo DESCARGADO del registry, deslogueado, sin compilar nada.
# Es lo que van a hacer los entornos de QA y produccion del TP6.

```

Las tres limpiezas y después el pull anónimo. El `builder prune` es el que nadie ve venir: acá liberó 40 GB de capas guardadas. Al final, el sistema completo levantado desde el registry — en el `ps` la columna IMAGE dice `ghcr.io/...`, no un nombre local.

Una advertencia honesta sobre arquitecturas. Estas imágenes se construyeron en una Mac con chip ARM, y lo verifiqué en vez de suponerlo: `docker manifest inspect` devuelve una sola plataforma real, `linux/arm64` (el `unknown/unknown` que aparece al lado es el manifiesto de atestación de BuildKit, no una arquitectura). Alguien con Intel/AMD recibiría `no matching manifest for linux/amd64` — y **los runners de GitHub Actions son Intel**, así que esto va a reaparecer en el TP7, donde se resuelve con `docker buildx`.

11. Problemas encontrados y aprendizajes

`npm ci` falló en el build: no había lockfile

Escribí el `package.json` a mano y fui directo al `docker build`, sin haber corrido nunca `npm install`. `npm ci` **exige** el `package-lock.json` — es justamente lo que lo hace reproducible, y por eso es el comando correcto para un build de contenedor. Se resolvió generando el lockfile y commiteándolo, que es donde tiene que estar.

`docker images` no mostraba las imágenes base

Docker 29 guarda las imágenes base de las etapas de build en el **cache de construcción**, no en el store de imágenes, así que el comando de comparación de la guía devolvía la tabla vacía. No estaba roto: hay que bajarlas explícitamente con `docker pull` para poder compararlas. El mensaje no sugiere nada de esto.

Los packages de ghcr nacen privados

Después del `push`, `gh api /user/packages` los mostraba como `private`. Mientras lo estén nadie puede hacer `docker pull`: ni la cátedra, ni otra máquina, ni un pipeline. Hay que cambiar la visibilidad desde la página del package, y hacerlo **para las dos** imágenes.

El cache se esconde en tres lugares

Para probar el pull anónimo hay que borrar: (1) las imágenes que construyó compose (`--rmi local`), (2) los nombres que uno les puso (`docker rmi`), y (3) el **cache de construcción** (`docker builder prune -af`). Si se saltea cualquiera de los tres, el `pull` contesta `Already exists` en todas las capas y no baja nada — y uno cree que probó algo que no probó.

Declaración de uso de Inteligencia Artificial

Usé **Claude Code (Claude Opus)**, declarado según el reglamento §6.

Qué fue asistido	Cómo lo verifiqué
El código de la aplicación	Probé las 7 reglas contra la API levantada , una por una, verificando código HTTP y mensaje: SKU duplicado → 400, borrar categoría con productos → 400, salida mayor al stock → 400, cantidad cero → 422, producto inactivo → 400, y el descuento en el borde exacto (9 sin, 10 con)
Dockerfiles, compose y <code>nginx.conf</code>	Los leí línea por línea y los probé: el sistema levanta con un comando, el front habla con el back por el proxy, y la prueba de persistencia se comporta como esperaba. Los tamaños los medí
La publicación en el registry	La verificación no es que la página diga <i>Public</i> : es que el <code>pull</code> funcione deslogueado , y que el sistema levante desde el registry sin compilar nada
Redacción de <code>decisiones.md</code> , <code>evidencias.md</code> y este informe	Los revisé para que digan lo que efectivamente pasó. Los problemas listados son los que me pasaron a mí

Lo que **no** delegué: la elección de la app y su dominio, las 7 reglas de negocio, la decisión de llamar a la API por ruta relativa en vez de URL absoluta, y qué persiste y qué no.

Anexo — Los comandos, en orden

```
# — Primera pasada: el sample de la cátedra (no se entrega) ———
git clone --branch demo-c2-inicio https://github.com/ingsoft3ucc/demo-fullstack.git practica-tp2
curl -fsSL https://dot.net/v1/dotnet-install.sh | bash -s -- --channel 8.0
docker run -d --name pg-tp2 -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=app -p 5432:5432 postgres:16-alpine
docker build -t mi-backend:dev ./backend
docker run --rm -d -p 8080:8080 -e ConnectionStrings__Default='Host=host.docker.internal;...' mi-backend:dev
docker compose up -d --build

# — Segunda pasada: mi app (esto SÍ se entrega) ———
cp .env.example .env
docker compose up -d --build
./scripts/datos-demo.sh

# — Las 7 reglas, probadas contra la API ———
curl -X POST localhost:8000/api/productos -d '{"sku":"BEB-001",...}' # → 400 SKU duplicado
curl -X DELETE localhost:8000/api/categorias/1 # → 400 tiene productos
curl -X POST localhost:8000/api/movimientos -d '{"cantidad":100,...}' # → 400 stock
insuficiente
curl -X POST localhost:8000/api/movimientos -d '{"cantidad":0,...}' # → 422 cantidad > 0

# — Persistencia ———
docker compose down && docker compose up -d # los datos SIGUEN
docker compose down -v && docker compose up -d # la lista vuelve VACÍA

# — Registry ———
gh auth refresh -h github.com -s write:packages
gh auth token | docker login ghcr.io -u marcosdon28 --password-stdin
docker tag ingesoft3-2319274-backend ghcr.io/marcosdon28/inventario-backend:v0.1.0
docker push ghcr.io/marcosdon28/inventario-backend:v0.1.0
# ... y hacerlas públicas desde Package settings → Change visibility

# — La prueba de verdad: pull sin credenciales ———
docker compose down --rmi local # 1. las imágenes que construyó compose
docker rmi ghcr.io/marcosdon28/inventario-backend:v0.1.0 # 2. los nombres propios
docker builder prune -af # 3. el CACHE DE CONSTRUCCIÓN (liberó 40 GB)
docker logout ghcr.io
docker pull ghcr.io/marcosdon28/inventario-backend:v0.1.0
docker compose -f docker-compose.registry.yml up -d # descarga, no construye

# — Cerrar el práctico ———
git tag -a v2.0.0 -m "TP2 cerrado" && git push origin v2.0.0
```